

MOBILE DEVELOPMENT

Consuming Web Services Using REST Apps



KHOA CÔNG NGHỆ THÔNG TIN
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

fit@hcmus

CONTENTS

- ☐ OVERVIEW
- ☐ CLIENT SIDE – CONSUMING WEBSERVICES
- ☐ EXAMPLES

OVERVIEW

- A WebService is a Consumer_Machine-to-Provider_Machine collaboration schema that operates over a computer network.
- The data exchanges occur independently of the OS, browser, platform, and programming languages used by the provider and the consumers.
- A provider may expose multiple EndPoints (sets of WebServices), each offering any number of typically related functions.
- WebServices expect the computer network to support standard Web protocols such as XML, HTTP, HTTPS, FTP, and SMTP.
- Example: Weather information, money exchange rates, world news, stock market quotation are examples of applications that can be modeled around the notion of a remote data-services provider surrounded by countless consumers tapping on the server's resources.

OVERVIEW

(Advantages of using the webService architecture)

- Under the WebService strategy the invoked functions are implemented once (in the server) and called many times (by the remote users).
- Some advantages of this organization are:
 - ▣ Elimination of redundant code,
 - ▣ Ability to transparently make changes on the server to update a particular service function without clients having to be informed.
 - ▣ Reduced maintenance and production costs.

OVERVIEW

(Why should Android developer learn how to create WebService?)

- Simple apps are usually self-contained and do not need to collaborate with other parties to obtain additional data or services (for instance, think of a scientific calculator)
- However, there are many cases in which the data needed to work with is very extensive, or changes very often and cannot (should not) be hard-coded into the app. Instead, this kind of data should be requested from a reliable external source (for instance, what is the Euro-to-Dollar rate of change right now?)
- Another class of apps requires a very high computing power perhaps not available in the mobile device (think of problems such as finding the shortest/fastest route between mapped locations, or best air-fare & route selection for a traveler)
- It is wise for an Android developer to learn how to solve typical problems that exceed the capacities of the handheld devices. Understanding the possibilities offered by the client-server computing model will make the developer be a more complete and better professional.

CLIENT SIDE – CONSUMING WEBSERVICES

- There are 2 widely used forms of invoking & consuming WebServices:
 - ▣ Representational State Transfer (REST): Closely tie to the HTTP protocol by associating its operation to the common methods: GET, POST, PUT, DELETE for HTTP/HTTPS. This model has a simple invocation mode and little overhead. Service calls rely on a URL which may also carry arguments. Sender & receiver must have an understanding of how they pass data items from one another. As an example: Google Maps API uses the REST model.
 - ▣ Remote Procedure Call (RPC): a software communication protocol that one program can use to request a service from a program located in another computer on a network without having to understand the network's details. Some implementations of this category include: Simple Object Access Protocol (SOAP), Common Object Request Broker Architecture (CORBA), Microsoft's Distributed Component Object Model (DCOM) and Sun Microsystems's Java/Remote Method Invocation (RMI).

CLIENT SIDE – CONSUMING WEBSERVICES

- Example: Using REST. The following URL is used to make a call to the Google Search service asking to provide links to the subject “Cleveland State University”

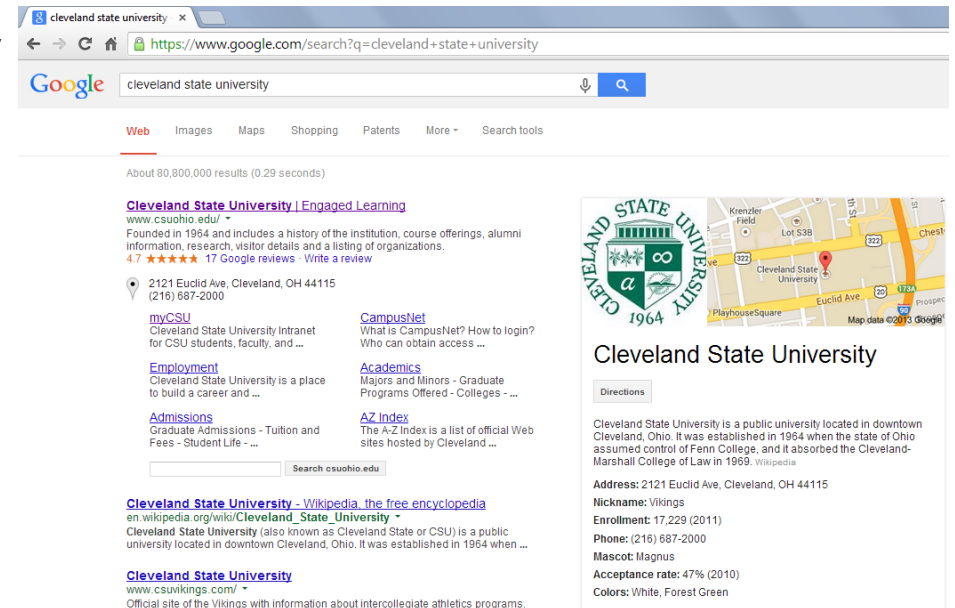
<https://www.google.com/search?q=cleveland+state+university>

Transport

Provider

Action

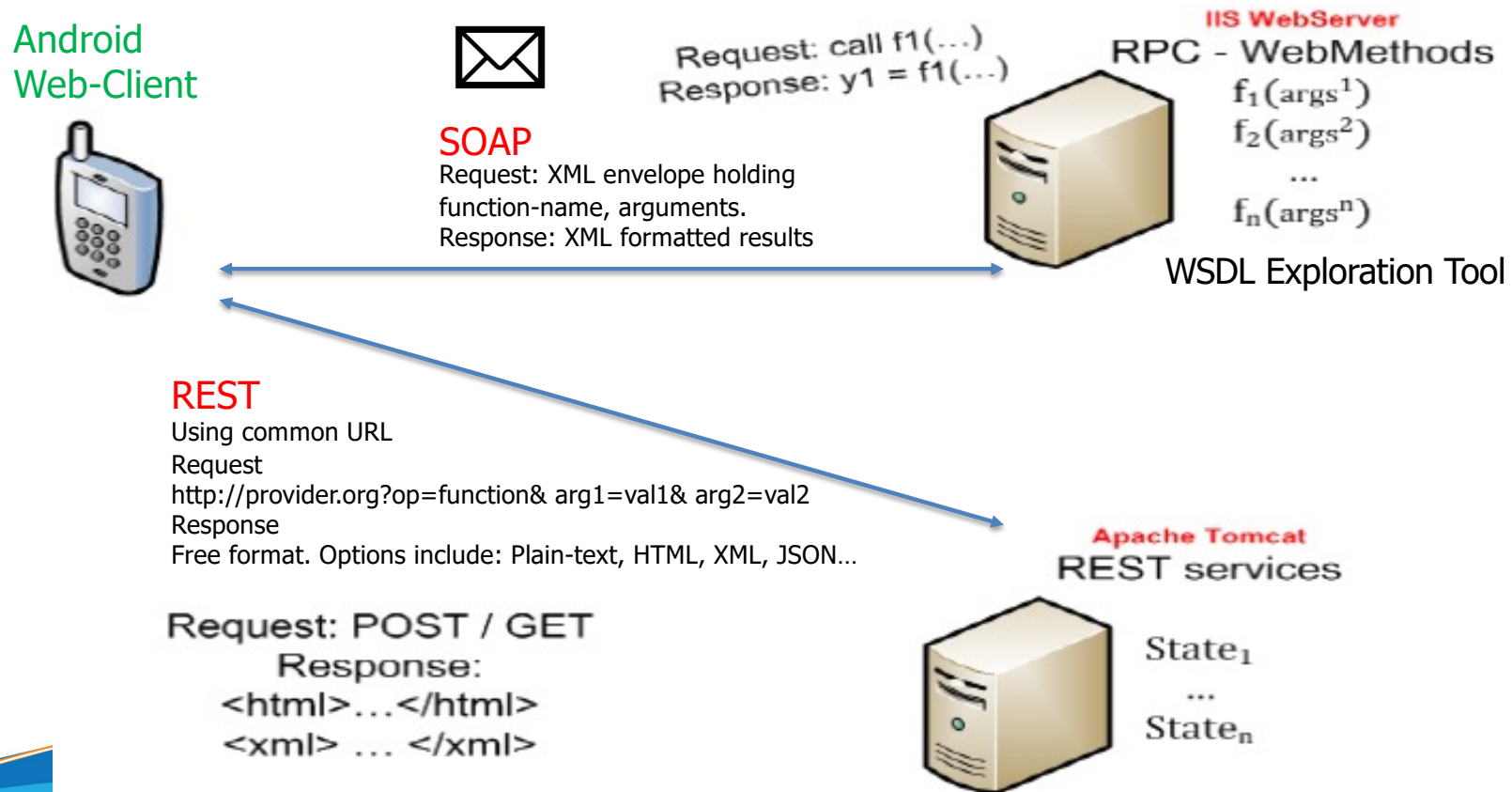
Arguments



CLIENT SIDE – CONSUMING WEBSERVICES (REST vs. SOAP)

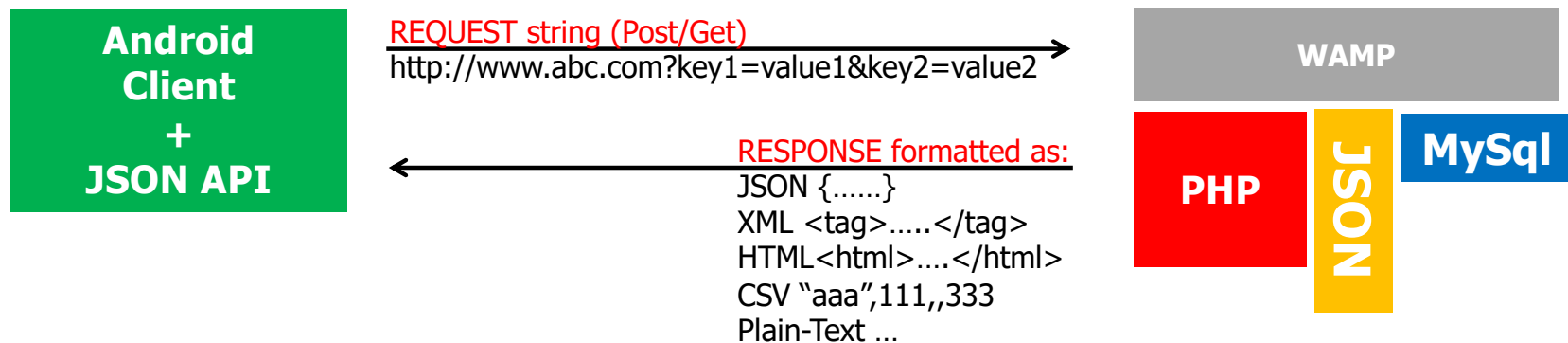
- Although SOAP and REST technologies accomplish the same final goal, that is request and receive a service, there are various differences between them.
 - REST users refer to their remote services through a conventional URL that commonly includes the location of the (stateless) server, the service name, the function to be executed and the parameters needed by the function to operate (if any). **Data is transported using HTTP/HTTPS.**
 - SOAP requires some scripting effort to create an XML envelop in which data travels. An additional overhead on the receiving end is needed to extract data from the XML envelope. **SOAP accepts** a variety of transport mechanisms, among them **HTTP, HTTPS, FTP, SMTP, etc.**
 - SOAP uses WSDL (WebService Description Language) for exposing the format and operation of the services. REST lacks an equivalent exploratory tool.

CLIENT SIDE – CONSUMING WEBSERVICES (WebClient consuming services using REST & SOAP)



REPRESENTATIONAL STATE TRANSFER (REST)

- In this second example an Android client uses REST protocol to interact with a WAMP Server. WebServices are implemented as a set of PHP programs. We use JSON encoding for the client-server data exchange.



REPRESENTATIONAL STATE TRANSFER

(Example 1: HttpURLConnection web-client)

- Call sample REST API and process response

- <https://api.sampleapis.com/codingresources/codingResources>

```
1  [  
2    {  
3      "id": 1,  
4      "description": "South Florida's Best Meetups on Youtube",  
5      "url": "https://www.youtube.com/bocajs",  
6      "types": ["tutorial"],  
7      "topics": ["js", "html"],  
8      "levels": ["beginner", "intermediate", "advanced"]  
9    },  
10   {  
11     "id": 2,  
12     "description": "BocaJS - South Florida's Largest Developer Meetup",  
13     "url": "https://Meetup.com/bocajs",  
14     "types": ["meeting"],  
15     "topics": ["js", "html"],  
16     "levels": ["beginner", "intermediate", "advanced"]  
17   }  
18 ]
```

REPRESENTATIONAL STATE TRANSFER

(Example 1: HttpURLConnection web-client)

- Add dependencies:

```
implementation 'com.fasterxml.jackson.module:jackson-module-kotlin:2.12.1'
```

- **Serializable** data class

```
data class ResourceInfo(  
    val id: String,  
    val description: String,  
    val url: String,  
    val types: List<String>,  
    val topics: List<String>,  
    val levels: List<String>  
)
```

REPRESENTATIONAL STATE TRANSFER

(Example 1: HttpURLConnection web-client)

□ Use AsyncTask

```
class MyAsync(context: Context) : AsyncTask<String, Int, Void>() {  
    var jsonResponse = ""  
    var onPostExecuteE:((String)->Unit)? = null  
  
    private val dialog = ProgressDialog(context)  
  
    override fun onPreExecute() {  
        this.dialog.setCancelable(false) //outside touch doesn't dismiss you  
        this.dialog.show()  
    }  
}
```

REPRESENTATIONAL STATE TRANSFER

(Example 1: HttpURLConnection web-client)

```
override fun doInBackground(vararg p0: String?): Void? {  
    try {  
        // STEP1. Create a HttpURLConnection object for GET method  
        val url = URL(p0[0])  
        val urlConnection = url.openConnection() as HttpURLConnection  
  
        urlConnection.requestMethod = "GET"  
        // True-indicates that the app will read data from the URL conn  
        urlConnection.setDoInput(true)  
        urlConnection.connect()  
        // STEP2. wait for incoming RESPONSE stream, place data in a buffer  
        val isResponse = urlConnection.getInputStream();  
        val responseBuffer = BufferedReader(InputStreamReader(isResponse))  
        // STEP3. Arriving JSON fragments are concatenate into a StringBuilder  
        val strBuilder = StringBuilder()  
        var myLine = responseBuffer.readLine()  
        while (myLine != null) {  
            strBuilder.append(myLine)  
            myLine = responseBuffer.readLine()  
        }  
    }  
}
```

```
        // show response (JSON encoded data)  
        jsonResponse = strBuilder.toString();  
        Log.i("nlhdung", "response:" + jsonResponse);  
    } catch (e :Exception){  
        Log.i("Error", e.message.toString())  
    }  
  
    return null  
}
```

REPRESENTATIONAL STATE TRANSFER

(Example 1: HttpURLConnection web-client)

```
override fun onPostExecute(result: Void?) {  
    super.onPostExecute(result)  
  
    //Update UI  
    if (this.dialog.isShowing())  
        this.dialog.dismiss();  
  
    // and log  
    val mapper = jacksonObjectMapper()  
    val resourceList: List<ResourceInfo> = mapper.readValue(jsonResponse)  
    println(resourceList.get(0))  
  
    // call back  
    onPostExecuteE?.invoke("Some other callback data")  
}
```

REPRESENTATIONAL STATE TRANSFER

(Example 2: Retrofit)

- Android Retrofit is simple to use. It essentially lets you treat API calls as simple Java/Kotlin method calls, so you only define which URLs to hit and the types of the request/response parameters as Java/Kotlin classes.
- The entire network call + JSON/XML parsing is completely handled by it (with help from Gson for JSON parsing), along with support for arbitrary formats with pluggable serialization/deserialization.

	One Discussion	Dashboard (7 requests)	25 Discussions
AsyncTask	941 ms	4,539 ms	13,957 ms
Volley	560 ms	2,202 ms	4,275 ms
Retrofit	312 ms	889 ms	1,059 ms

REPRESENTATIONAL STATE TRANSFER

(Example 2: Retrofit)

- ☐ Call sample REST API and process response
- ☐ Sample API
 - ☒ <https://api.sampleapis.com/codingresources/codingResources>

REPRESENTATIONAL STATE TRANSFER

(Example 2: Retrofit)

□ Add dependencies:

```
implementation 'com.squareup.retrofit2:retrofit:2.9.0'  
implementation 'com.squareup.retrofit2:converter-gson:2.9.0'
```

□ **Serializable** data class

```
data class ResourceInfo(  
    val id: String,  
    val description: String,  
    val url: String,  
    val types: List<String>,  
    val topics: List<String>,  
    val levels: List<String>  
)
```

REPRESENTATIONAL STATE TRANSFER

(Example 2: Retrofit)

```
import retrofit2.Call
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import retrofit2.http.GET

interface ApiInterface
{
    @GET("codingResources")
    fun getResources() : Call<List<ResourceInfo>>

    companion object {
        var BASE_URL = "https://api.sampleapis.com/codingresources/"

        fun create() : ApiInterface {
            val retrofit = Retrofit.Builder().addConverterFactory(GsonConverterFactory.create())
                .baseUrl(BASE_URL).build()
            return retrofit.create(ApiInterface::class.java)
        }
    }
}
```

REPRESENTATIONAL STATE TRANSFER

(Example 2: Retrofit)

```
val dialog = ProgressDialog(this)
dialog.setCancelable(false) //outside touch doesn't dismiss you
dialog.show()

val apiInterface = ApiInterface.create().getResources()
apiInterface.enqueue( object : Callback<List<ResourceInfo>> {
    override fun onResponse(call: Call<List<ResourceInfo>>?, response: Response<List<ResourceInfo>>?) {
        dialog.dismiss()

        if(response?.body() != null) {
            var resourceList: List<ResourceInfo> = response.body()!!
            Log.i("nlhdung", resourceList.get(0).toString())
        }
    }

    override fun onFailure(call: Call<List<ResourceInfo>>, t: Throwable) {
        Log.i("nlhdung", "onFailure"); //t.message
    }
})
```

Practice

- Given some free api:
 - <https://jsonplaceholder.typicode.com>
 - <https://developers.thecatapi.com/view-account/ylX4blBYT9FaoVd6OhvR?report=bOoHBz-8t>
- Use the example 1,2,3 in 'A10.a-User Interfaces - Menus, Action bar and Tab' to load 2 tabs:
 - Tab1: show list of information from jsonplaceholder
 - Tab2: show list of cats (with picture) from thecatapi

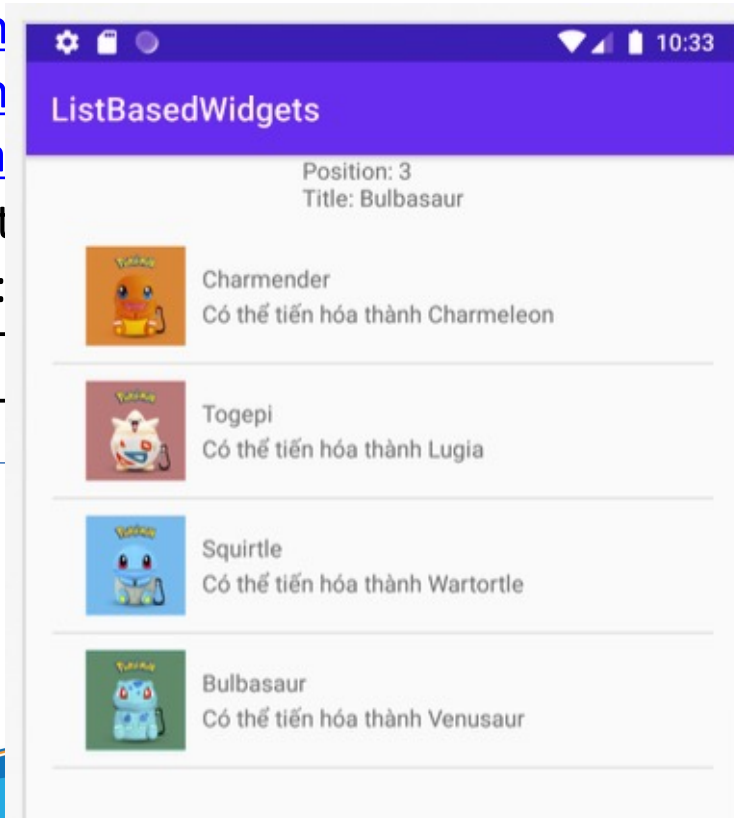
Practice

- Given some free api:

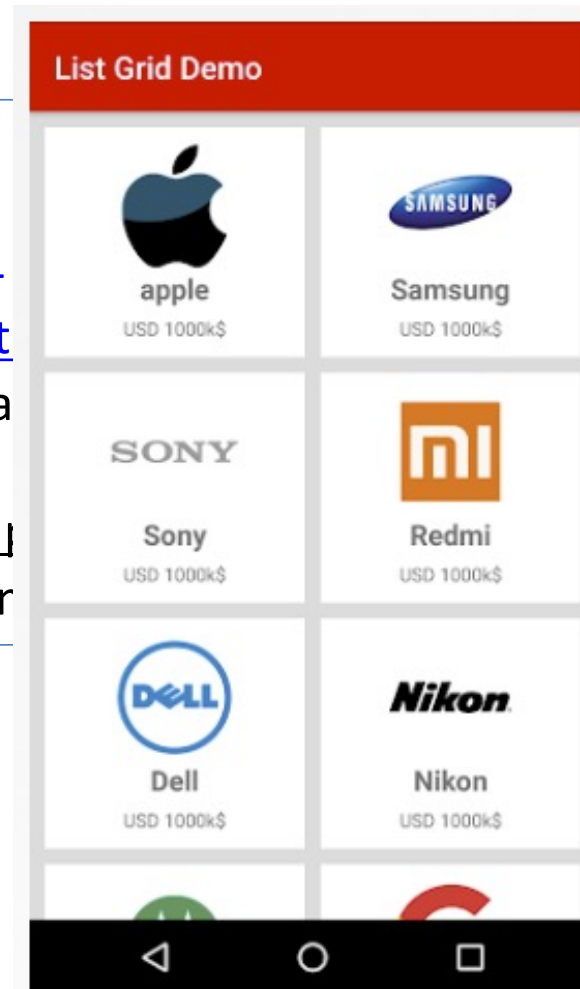
- [h](#)
- [h](#)
- [a](#)

- Use tabs:

- T
- T



om
view-
eport
nterfa
n json
e) from



d Tab' to load 2